PROMETEO · by thePower

MPO I CMO-313 FUNDAMENTOS DE PROGRAMACIÓN - DAM · DAM

Unidad 1: Ampliación en introducción a la programación

UNIDAD 1

Índice de contenidos

Objetivos de la unidad 3

Introducción 4

1. Paradigmas de programación 4

1.1. Qué es un paradigma y sus tipos

1.2. Pensamiento multiparadigma

2. Eficiencia y buenas prácticas 6

2.1. Eficiencia algorítmica: la notación Big-O

2.2. Diseño incremental, refactorización y Clean Code

Ampliación y actualización 8

Mini proyecto guiado 10

Ejercicio 10

Resumen profesionalizante 10

RESULTADOS DE APRENDIZAJE

Objetivos de la unidad

Al finalizar esta unidad, serás capaz de:

- Comprender qué es un paradigma de programación.
- Distinguir los paradigmas imperativo, declarativo, funcional y lógico.
- Adoptar un pensamiento multiparadigma.
- Valorar la eficiencia algorítmica con la notación Big-O.
- Aplicar el diseño incremental y las buenas prácticas (Clean Code).

ACTIVACIÓN Y REFLEXIÓN

Introducción

Preguntas para reflexionar:

- ¿Existe una única forma de programar o hay varios enfoques (paradigmas)?
- ¿Cómo se mide si un algoritmo es eficiente?
- ¿Qué distingue un código profesional de uno que «solo funciona»? Este **módulo optativo (MPO I, «Ampliación de Programación», CMO-313)** amplía y profundiza el módulo **0485 Programación** con contenidos de **nivel profesional**. Esta primera unidad eleva la mirada sobre la programación: cubre los **paradigmas de programación** (qué es un paradigma; imperativo, declarativo, funcional y lógico; pensamiento multiparadigma) y la **eficiencia y las buenas prácticas** (eficiencia algorítmica con **Big-O**; diseño incremental y metodologías ágiles; refactorización y **Clean Code**). Un buen desarrollador no solo escribe código que funciona: elige el **enfoque** adecuado, valora la **eficiencia** y aplica **buenas prácticas**. Culmina en el análisis de un problema resuelto con distintos paradigmas y la evaluación Big-O de dos algoritmos.

Desarrolla los **paradigmas de programación** (qué es un paradigma; imperativo, declarativo, funcional y lógico; pensamiento multiparadigma) y la **eficiencia y buenas prácticas** (Big-O; diseño incremental y ágil; refactorización y Clean Code). El **análisis de un problema con distintos paradigmas y la evaluación Big-O de dos algoritmos** es el entregable. Es qué enfoques de programación existen y cómo se valora la calidad y la eficiencia del código.



distintos paradigmas y enfoques de programación

1. Paradigmas de programación

1.1. Qué es un paradigma y sus tipos

Un **paradigma de programación** es un **enfoque** o «estilo» de programar: una forma de pensar y estructurar las soluciones. No hay una única manera de resolver un problema con código; conocer los paradigmas amplía las herramientas mentales del desarrollador:

- **Qué es un paradigma:** un modelo o filosofía sobre **cómo** organizar el código y el razonamiento para resolver problemas. Un mismo problema puede resolverse con distintos paradigmas.
- **Imperativo:** se describe **paso a paso** *cómo* hacer las cosas, con instrucciones que cambian el estado (variables, bucles). Es el más clásico (incluye el **estructurado** y el **orientado a objetos**).
- **Declarativo:** se describe **qué** se quiere obtener, no cómo (el sistema decide el cómo). Por ejemplo, **SQL** («dame los productos con precio > 100») o HTML.
- **Funcional:** se basa en **funciones** (sin efectos secundarios) que se combinan; los datos son **inmutables**. En Java, las **lambdas** y **Streams** aportan este estilo.
- **Lógico:** se describen **hechos y reglas** y el sistema **deduce** las respuestas (Prolog). Menos común, pero potente en IA y razonamiento.



EL MISMO PROBLEMA: QUEDARSE CON LOS PRODUCTOS DE MÁS DE 100 €

Imperativo | Declarativo | Funcional

```java
// IMPERATIVO: se dice paso a paso cómo hacerlo
List<Producto> caros = new ArrayList<>();
for (Producto p : productos) {
    if (p.getPrecio() > 100) caros.add(p);
}

// FUNCIONAL: se combinan funciones sobre un flujo de datos
var caros2 = productos.stream()
        .filter(p -> p.getPrecio() > 100)
        .toList();
```

los cuatro paradigmas (imperativo, declarativo, funcional, lógico) y su enfoque

1.2. Pensamiento multiparadigma

Los lenguajes modernos —como **Java**— son **multiparadigma**: permiten combinar varios enfoques. Un buen desarrollador no se casa con uno solo, sino que **elige el más adecuado** para cada parte del sistema. Es el «pensamiento multiparadigma»:

- **Java es multiparadigma:** es orientado a objetos (imperativo), pero incorpora el estilo **funcional** (lambdas, Streams) y se combina con el **declarativo** (SQL en el acceso a datos).
- **Elegir el enfoque por capa del sistema:** cada parte de una aplicación puede beneficiarse de un paradigma:
- La **lógica de negocio** con objetos (POO).
- El **procesamiento de datos** con estilo **funcional** (map/filter/reduce sobre Streams).
- El **acceso a datos** con **SQL** declarativo.

- **Ventaja:** combinar paradigmas produce un código más **claro, conciso y mantenible** que forzar todo a un único estilo. **Caso de estudio: Spotify y el enfoque adecuado por capa.** Una plataforma como **Spotify** combina paradigmas: la lógica de las cuentas y las listas con **objetos**, el procesamiento de las reproducciones y recomendaciones con estilo **funcional** (transformar grandes flujos de datos con map/filter), y las consultas a la base de datos de forma **declarativa**. Elegir el paradigma por capa hace el sistema más eficiente y mantenible.

Elegir el paradigma por capa del sistema



elección de paradigma por capa (POO en negocio, funcional en datos, SQL en persistencia)

En la práctica. Un paradigma es un enfoque de programar; conoce el imperativo, declarativo, funcional y lógico. Java es multiparadigma: combina POO con estilo funcional (lambdas/Streams) y SQL declarativo. Elige el paradigma por capa: objetos en el negocio, funcional en los datos, SQL en la persistencia. No fuerces todo a un solo estilo: combinar paradigmas mejora la claridad y el mantenimiento.

Un error frecuente es pensar que la programación orientada a objetos es el único paradigma que importa. En realidad, la POO es fundamental, pero los lenguajes modernos son multiparadigma. El estilo funcional (Streams), el declarativo (SQL) y otros aportan mucho según el problema. Conocer varios paradigmas hace a un mejor programador.

Un error frecuente es pensar que elegir el paradigma es una cuestión teórica sin efecto práctico. En realidad, el paradigma adecuado hace el código más claro, conciso y eficiente. Procesar datos con Streams (funcional) o consultar con SQL (declarativo) es mucho mejor que hacerlo todo con bucles imperativos. La elección tiene efecto real.

SECCIÓN 2

2. Eficiencia y buenas prácticas

2.1. Eficiencia algorítmica: la notación Big-O

No basta con que un algoritmo funcione: debe ser **eficiente**. La **notación Big-O** describe cómo **crece** el coste (tiempo o memoria) de un algoritmo a medida que crecen los datos. Es fundamental para valorar y comparar soluciones:

- **Qué mide Big-O:** el **orden de crecimiento** del coste según el tamaño de la entrada n (no el tiempo exacto, sino cómo escala).
- **Complejidades habituales (de mejor a peor):**
- **O(1)** — constante: el coste **no** depende de n (acceder a un elemento de un array).
- **O(log n)** — logarítmica: muy eficiente (búsqueda binaria).
- **O(n)** — lineal: el coste crece proporcional a n (recorrer una lista).
- **O(n²)** — cuadrática: crece muy rápido (dos bucles anidados); mala para datos grandes.
- **Por qué importa:** con pocos datos, la diferencia es imperceptible; con **millones**, un O(n²) puede tardar horas donde un O(n log n) tarda segundos. **Caso de estudio: Amazon y la escala.** En **Amazon**, con millones de productos y usuarios, la eficiencia algorítmica es crítica: una búsqueda debe ser O(log n) o mejor, no O(n²). Elegir algoritmos y estructuras eficientes (Big-O bajo) es lo que permite responder en milisegundos a esa escala.

Complejidad	Cómo crece el coste	Ejemplo	Operaciones con n = 10	Con n = 1.000	Con n = 1.000.000
O(1) · constante	El coste **no depende** de n	Acceder a un elemento de un array	1	1	1
O(log n) · logarítmica	Muy eficiente: al doblar los datos solo añade un paso	Búsqueda binaria	≈ 3	≈ 10	≈ 20
O(n) · lineal	Proporcional a n	Recorrer una lista	10	1.000	1.000.000
O(n²) · cuadrática	Crece **muy rápido**; mala para datos grandes	Dos bucles anidados	100	1.000.000	1.000.000.000.000

2.2. Diseño incremental, refactorización y Clean Code

Un código profesional se construye de forma **incremental** y se mantiene **limpio**. Estas son las buenas prácticas que distinguen el código profesional del que «solo funciona»:

- **Diseño incremental y metodologías ágiles:** construir el software **poco a poco**, en **iteraciones**, empezando por un **MVP (producto mínimo viable)** que funcione y ampliándolo. Se entrega valor pronto y se adapta al cambio (visto en 0487). Por ejemplo, **Trello** creció así: primero lo esencial (tableros y tarjetas), luego funciones avanzadas.
- **Refactorización y Clean Code:** mejorar continuamente el código para que sea **limpio** (legible, simple, bien nombrado), aplicando los principios del **Clean Code** (nombres claros, funciones cortas, sin duplicación). Es la optimización vista en 0487, aquí a nivel profesional.
- **Análisis estático:** herramientas como **SonarQube** o **Checkstyle** analizan el código automáticamente y detectan problemas de calidad, *code smells* y vulnerabilidades. Por ejemplo, **BBVA** usa análisis estático para garantizar la calidad de su software bancario. **Caso de estudio: BBVA y la calidad continua. BBVA**, con software crítico bancario, aplica **análisis estático** (SonarQube) y **Clean Code** de forma sistemática: cada

cambio se analiza automáticamente para mantener la calidad y la seguridad. Muestra cómo las buenas prácticas son imprescindibles en el software profesional.

Comprender el **diseño incremental** y el **Clean Code** completa las buenas prácticas y es el entregable. Comprender qué distingue un código profesional de uno que solo funciona es esencial.

Un código profesional se construye de forma incremental y se mantiene limpio: el diseño incremental y las metodologías ágiles construyen el software poco a poco, en iteraciones, empezando por un MVP que funcione y ampliándolo (como creció Trello); la refactorización y el Clean Code mejoran continuamente el código para que sea limpio (legible, simple, bien nombrado), aplicando los principios de nombres claros, funciones cortas y sin duplicación; y el análisis estático con SonarQube o Checkstyle analiza el código automáticamente y detecta problemas de calidad, code smells y vulnerabilidades, como hace BBVA en su software bancario para garantizar la calidad y la seguridad.

En la práctica. Usa Big-O para valorar la eficiencia: $O(1) < O(\log n) < O(n) < O(n^2)$. Construye de forma incremental (MVP + iteraciones); entrega valor pronto. Aplica el Clean Code: nombres claros, funciones cortas, sin duplicación. Apóyate en el análisis estático (SonarQube, Checkstyle) para vigilar la calidad automáticamente.

Un error frecuente es pensar que si el algoritmo da el resultado correcto, su eficiencia (Big-O) da igual. En realidad, con datos grandes, la eficiencia es crítica: un $O(n^2)$ puede volver una aplicación inservible donde un $O(n \log n)$ responde al instante. Valorar el Big-O es parte de escribir buen software.

Un error frecuente es pensar que el Clean Code y el análisis estático son lujos para grandes empresas. En realidad, las buenas prácticas benefician a cualquier proyecto: hacen el código mantenible y con menos errores. Empresas como BBVA las aplican por necesidad, pero son valiosas a cualquier escala.

Ampliación y actualización

Programación funcional en Java y arquitecturas limpias

El paradigma **funcional** ha dejado de ser algo exótico para convertirse en parte del **código Java cotidiano**. Las **lambdas** (funciones anónimas) y la **API Streams** permiten procesar colecciones de forma **declarativa** (`map`, `filter`, `reduce`) en lugar de con bucles imperativos, produciendo un código más **conciso y legible**:

```
// Estilo funcional con Streams: filtrar y transformar una lista
List<String> caros = productos.stream()
    .filter(p → p.getPrecio() > 100)   // solo los caros
    .map(Producto::getNombre)          // quedarse con el nombre
    .toList();
```

Además, para organizar sistemas grandes combinando paradigmas, se usan las **arquitecturas limpias** como la **hexagonal (puertos y adaptadores)**, que separan la **lógica de negocio** (el núcleo) de los detalles técnicos (la base de datos, la interfaz), conectándolos mediante **puertos** y **adaptadores**. Así cada capa puede usar el paradigma que mejor le convenga y el sistema es más **mantenible y testeable**. Para el desarrollador DAM, conocer la **programación funcional en Java y las arquitecturas limpias** eleva su nivel profesional. Refleja cómo la programación funcional (lambdas, Streams) y las arquitecturas limpias (hexagonal) organizan el código moderno.

MOCKUP DE DOCUMENTO

programación funcional en Java (lambdas, Streams) y arquitectura hexagonal (puertos y adaptadores)

programación funcional en Java (lambdas, Streams)

arquitectura hexagonal (puertos y adaptadores)

programación funcional en Java (lambdas, Streams) y arquitectura hexagonal (puertos y adaptadores)

La IA como par de programación

Una de las mayores transformaciones recientes es el uso de la **IA como par de programación (pair programming)**. Asistentes como **GitHub Copilot** —integrados en el IDE— **sugieren y generan código**, proponen implementaciones, explican fragmentos y ayudan a aplicar patrones y buenas prácticas. Funcionan como un «compañero» que aporta ideas y acelera el trabajo, especialmente en tareas repetitivas o al explorar una API nueva.

Ahora bien, la clave es la **revisión crítica**: la IA acelera, pero el desarrollador debe **entender, evaluar y validar** lo que genera, especialmente el **diseño** (¿es el paradigma adecuado?, ¿es eficiente en Big-O?, ¿es limpio?). La IA no sustituye el criterio profesional: lo potencia si se usa con juicio. Para el desarrollador DAM, saber usar la **IA como par de programación con revisión crítica** es ya una competencia esencial. Refleja cómo la IA (Copilot) aumenta la productividad siempre con la revisión crítica del diseño por parte del desarrollador.



la IA como par de programación (Copilot) con revisión crítica del diseño

Mini proyecto guiado

Análisis multiparadigma y Big-O. Ampliación en introducción a la programación.

Paso 1: Explica los **paradigmas** (imperativo, declarativo, funcional, lógico) con un ejemplo de cada uno.

Paso 2: Resuelve un mismo problema con estilo **imperativo** (bucles) y **funcional** (Streams), y compáralos.

Paso 3: Evalúa el **Big-O** de dos algoritmos (por ejemplo, búsqueda lineal O(n) vs binaria O(log n)).

Paso 4: Aplica las **buenas prácticas** (Clean Code) y justifica la elección más eficiente.

Entregable: análisis de un problema resuelto con distintos paradigmas y evaluación Big-O de dos algoritmos.

Ejercicio

Analiza los paradigmas y la eficiencia de una solución.

Requisitos:

- Explica los paradigmas de programación y el pensamiento multiparadigma.
- Resuelve un problema con distintos paradigmas (imperativo y funcional).
- Evalúa el Big-O de dos algoritmos y justifica cuál es más eficiente.
- Aplica las buenas prácticas (diseño incremental, Clean Code, análisis estático).

Resumen profesionalizante

Con este conocimiento, ahora estás preparado para:

- Comprender qué es un paradigma de programación.
- Distinguir los paradigmas imperativo, declarativo, funcional y lógico.
- Adoptar un pensamiento multiparadigma.
- Valorar la eficiencia algorítmica con la notación Big-O.
- Aplicar el diseño incremental y las buenas prácticas (Clean Code).